

Universidad del Valle de Guatemala
Facultad de Ingeniería — CC3071 Diseño de Lenguajes de Programación

YALex & YAPar

Generador de Analizadores Léxicos y Sintácticos
Tablas de parseo SLR(1) y LALR(1)

Integrantes

Ian Cumes — 23236

Javier Cifuentes — 23079

Javier López — 28415

Informe técnico — implementación, algoritmos, validación y uso

Resumen ejecutivo

Se construyó un ecosistema completo de generación de analizadores: **YALex** (léxico, a partir de expresiones regulares) y **YAPar** (sintáctico, a partir de una gramática libre de contexto). YAPar implementa **dos** métodos de análisis ascendente, **SLR(1)** y **LALR(1)**, incluyendo la construcción del autómata LR(0), el cálculo de FIRST/FOLLOW, las tablas de parseo, un motor de parseo con traza paso a paso, generación de un parser autónomo, visualización del autómata y reporte de errores sintácticos y gramaticales. La implementación se validó contra el documento manual de la actividad (autómata de 12 estados, tabla y traza idénticas) y con 51 pruebas automáticas.

1. Introducción y objetivos

Un compilador procesa el código fuente en fases. La **fase léxica** divide el texto en *tokens* y la **fase sintáctica** verifica que esos tokens formen estructuras válidas según una gramática. Este proyecto genera ambos analizadores y, para el sintáctico, implementa SLR(1) y LALR(1) para poder compararlos.

- Producir tokens desde una especificación YALex.
- Construir el autómata LR(0) con CLOSURE, GOTO, FIRST y FOLLOW.
- Construir las tablas de parseo SLR(1) y LALR(1).
- Evaluar cadenas y determinar su corrección sintáctica.
- Reportar errores sintácticos y gramaticales durante el proceso.

2. Conceptos fundamentales (glosario)

Análisis ascendente (bottom-up): el parser lee los tokens de izquierda a derecha manteniendo una *pila*; reconstruye el árbol de derivación de las hojas hacia la raíz aplicando dos operaciones:

- **shift (desplazar):** introduce el token actual en la pila.
- **reduce (reducir):** reconoce que la cima de la pila es el cuerpo de una producción $A \rightarrow \beta$ (el **handle**) y lo sustituye por A.
- **accept:** la entrada se redujo al símbolo inicial $\rightarrow$ cadena válida.

Término	Definición
Terminal	Token producido por el lexer (MAYÚSCULAS): SENTENCE, AND...
No-terminal	Variable de la gramática (minúsculas): s, p, q...
Producción	Regla cabeza $\rightarrow$ cuerpo, p. ej. $s \rightarrow s \text{ AND } p$
Item LR(0)	Producción con un punto: $s \rightarrow s \cdot \text{AND } p$
Gramática aumentada	Se añade $S' \rightarrow S$ para un único estado de aceptación
FIRST(α)	Terminales con que puede empezar lo derivable de α
FOLLOW(A)	Terminales que pueden seguir a A
Prefijo viable	Prefijo de una forma sentencial que puede estar en la pila
Conflicto	Una celda de la tabla pide dos acciones (shift/reduce o reduce/reduce)

SLR(1) vs LALR(1): ambos usan los mismos estados (los del autómata LR(0)); difieren en *cuándo reducen*. SLR reduce $A \rightarrow \beta$ ante todo **FOLLOW(A)**; LALR reduce solo ante el **lookahead propio** del item (más preciso). Esto define la jerarquía **SLR** $\subset$ **LALR** $\subset$ **LR(1)**: cada método acepta más gramáticas que el anterior (se demuestra en §13–14).

3. Arquitectura general

El flujo es una tubería de cuatro etapas. La frontera entre YALex y YAPar es el **contrato de tokens**: el lexer expone `Lexer(text).tokenize()`, que entrega `Token(type, lexeme, line, column)`; YAPar usa `type` como terminal.

```
.yal --[YALex]--> lexer.py --tokenize()--> [Token, Token, ...]
                                     |
.yalp --[YAPar]--> Grammar --> aumentar --> FIRST / FOLLOW
                                     |
                                     +--> Automata LR(0)    --> Tabla SLR(1)
                                     +--> Automata LALR(1) --> Tabla LALR(1)
                                     |
[Token, ...] --> Motor de parseo (shift/reduce) --> ACEPTA / RECHAZA
                                                (+ traza, arbol, errores)
```

Módulo (yapargen/)	Responsabilidad
yapar_reader	Lee %token, IGNORE y el separador %%
yapar_parser	Producciones → Grammar; valida errores gramaticales
grammar	Production, Grammar, aumentación, orden de símbolos
first_follow	FIRST y FOLLOW por punto fijo
lr0_items	Item LR(0), CLOSURE, GOTO
lr0_automaton	Colección canónica LR(0) (BFS)
slr_table	Tabla SLR(1) e impresión
lr1_items / lalr_automaton / lalr_table	Items LR(1), fusión de núcleos, tabla LALR(1)
parse_runner / slr_parser	Motor shift/reduce con traza y árbol
diagnostics	Reporte de conflictos
visualize	Render del autómata (.dot/.png/.txt) y del árbol
codegen	Emite un parser .py autónomo
pipeline	Orquesta el análisis completo (analyze)

4. El archivo .yalp y la gramática

El archivo tiene una sección de tokens (%token, IGNORE) y, tras %, las producciones. Minúsculas = no-terminales; MAYÚSCULAS = terminales. El símbolo inicial es la primera producción.

```
/* ----- */
/* Gramatica SLR canonica de la actividad "Construccion del Automata */
/* LR(0)". Equivalencias con la notacion matematica del enunciado: */
/*   s = S      p = P      q = Q                                     */
/*   AND = ^    OR = v    LBRACKET = [   RBRACKET = ]   SENTENCE = sentence */
/* ----- */

/* INICIA Seccion de TOKENS */
%token AND OR
%token LBRACKET RBRACKET
%token SENTENCE
/* FINALIZA Seccion de TOKENS */

%%

/* INICIA Seccion de PRODUCCIONES */
S:
    s AND p
    | p
;

p:
```

```

        p OR q
    | q
;

q:
    LBRACKET s RBRACKET
    | SENTENCE
;
/* FINALIZA Seccion de PRODUCCIONES */

```

Gramática aumentada (la regla 0 es la de aceptación):

```

(0) s' -> s    <- aceptacion
(1) s -> s AND p
(2) s -> p
(3) p -> p OR q
(4) p -> q
(5) q -> LBRACKET s RBRACKET
(6) q -> SENTENCE

```

5. FIRST y FOLLOW

Se calculan por iteración de punto fijo hasta que los conjuntos no cambian. FIRST de una secuencia agrega FIRST del primer símbolo y, si es anulable, continúa con el siguiente; ϵ aparece si toda la secuencia es anulable.

Algoritmo FIRST

```
para cada terminal t: FIRST(t) = { t }
para cada no-terminal A: FIRST(A) = {}
repetir hasta que no haya cambios:
  para cada produccion A -> X1 X2 ... Xn:
    si es A -> e: agregar e a FIRST(A)
    si no: FIRST(A) |= FIRST(X1 X2 ... Xn)
```

Algoritmo FOLLOW

```
FOLLOW(simbolo_inicial) = { $ }
repetir hasta que no haya cambios:
  para cada produccion A -> a B b (B no-terminal):
    FOLLOW(B) |= FIRST(b) \ { e }
    si e in FIRST(b): FOLLOW(B) |= FOLLOW(A)
```

Resultado para la gramática de ejemplo:

```
FIRST
s: { LBRACKET, SENTENCE }
p: { LBRACKET, SENTENCE }
q: { LBRACKET, SENTENCE }

FOLLOW
s: { $, AND, RBRACKET }
p: { $, AND, OR, RBRACKET }
q: { $, AND, OR, RBRACKET }
```

6. Items LR(0), CLOSURE y GOTO

CLOSURE(I)

```
result = I
repetir hasta estabilizar:
  para cada item [A -> a . B b] en result (B no-terminal):
    para cada produccion B -> g:
      agregar [B -> . g] a result
```

GOTO(I, X)

```
J = { [A -> a X . b] por cada [A -> a . X b] en I }
devolver CLOSURE(J)
```

Ejemplo paso a paso. El estado inicial es CLOSURE($\{s' \rightarrow \cdot s\}$). Como el punto precede a s (no-terminal), se agregan sus producciones; luego, como aparece p y q tras un punto, también las de ellos. El resultado es el estado I0:

```
I0 = CLOSURE({s' -> . s})
(inicial):
p -> . p OR q
p -> . q
q -> . LBRACKET s RBRACKET
q -> . SENTENCE
s -> . p
s -> . s AND p
```

Y un paso de GOTO: GOTO(I0, s) mueve el punto sobre s y cierra, dando I1:

```
I1 = GOTO(I0, s)
s -> s . AND p
s' -> s .
```

7. Autómata LR(0)

La colección canónica se construye por BFS desde I0, generando un estado por cada conjunto de items alcanzable vía GOTO. La numeración de estados es determinista (no-terminales por orden de definición, luego terminales por primera aparición), por lo que coincide con el documento manual: **12 estados** (I0–I11).

Autómata LR(0) — gramática SLR

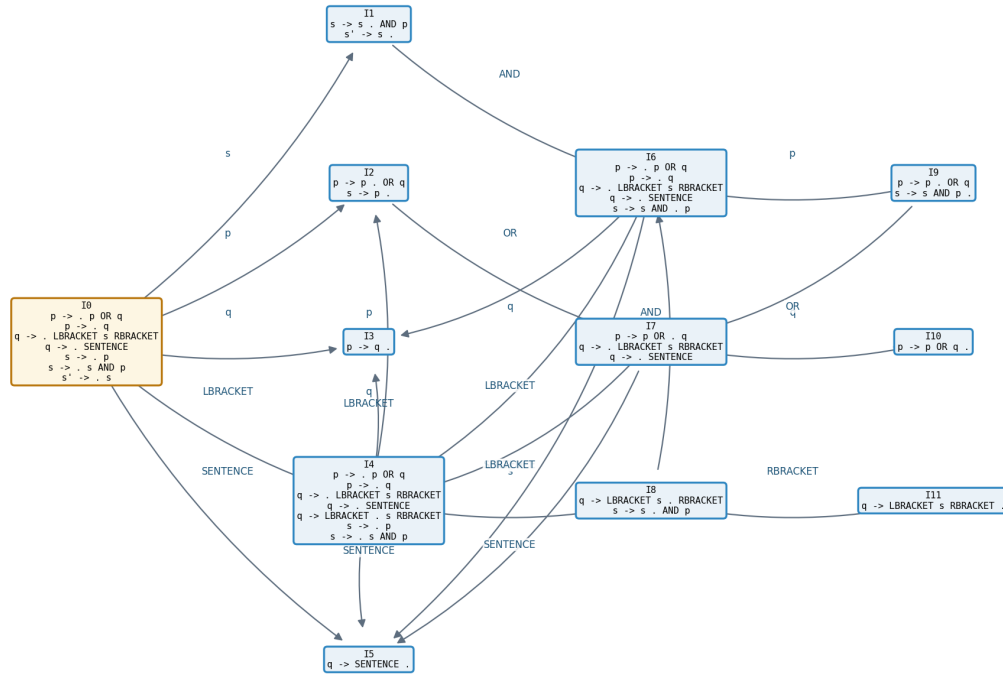


Figura 1. Autómata LR(0) de la gramática SLR (12 estados).

8. Tabla de parseo SLR(1)

Construcción de la tabla SLR(1)

```
para cada estado i con conjunto de items I_i:
  para [A -> a . x b] en I_i, x terminal y GOTO(I_i,x)=I_j:
    ACTION[i, x] = shift j
  para [A -> a .] en I_i (A != S'):
    para cada t en FOLLOW(A): ACTION[i, t] = reduce A->a
  para [S' -> S .] en I_i: ACTION[i, $] = accept
  para cada no-terminal A con GOTO(I_i,A)=I_j: GOTO[i, A] = j
```

La tabla generada coincide celda por celda con el oráculo de la actividad (s# = shift, r# = reduce por la regla #, acc = accept):

Tabla de parseo SLR(1)

Estado	AND	OR	LBRACKET	RBRACKET	SENTENCE	\$	s	p	q
0			s4		s5		1	2	3
1	s6					acc			
2	r2	s7		r2		r2			
3	r4	r4		r4		r4			
4			s4		s5		8	2	3
5	r6	r6		r6		r6			
6			s4		s5			9	3
7			s4		s5				10
8	s6			s11					
9	r1	s7		r1		r1			
10	r3	r3		r3		r3			
11	r5	r5		r5		r5			

Reglas (r#):

```
accept (S' -> S): s' -> s
r1: s -> s AND p
r2: s -> p
r3: p -> p OR q
r4: p -> q
r5: q -> LBRACKET s RBRACKET
r6: q -> SENTENCE
```

9. Motor de parseo y análisis de cadenas

Motor LR dirigido por tabla

```
pila = [0]; ip = 0 (apunta al token actual, con $ al final)
repetir:
  s = cima(pila); a = entrada[ip]; act = ACTION[s, a]
  si act = shift t: apilar t; ip += 1
  si act = reduce A->b: desapilar |b| estados; t = cima(pila)
  apilar GOTO[t, A]

  si act = accept: ACEPTAR
  si act vacio: ERROR sintactico (token inesperado)
```

Traza para sentence $\vee$ sentence (tokens SENTENCE OR SENTENCE), idéntica a la del documento manual:

Stack	Input	Acción
0	SENTENCE OR SENTENCE \$	shift -> I5
0 SENTENCE 5	OR SENTENCE \$	reduce r6: q -> SENTENCE
0 q 3	OR SENTENCE \$	reduce r4: p -> q
0 p 2	OR SENTENCE \$	shift -> I7
0 p 2 OR 7	SENTENCE \$	shift -> I5
0 p 2 OR 7 SENTENCE 5	\$	reduce r6: q -> SENTENCE
0 p 2 OR 7 q 10	\$	reduce r3: p -> p OR q
0 p 2	\$	reduce r2: s -> p
0 s 1	\$	accept

CADENA ACEPTADA ✓

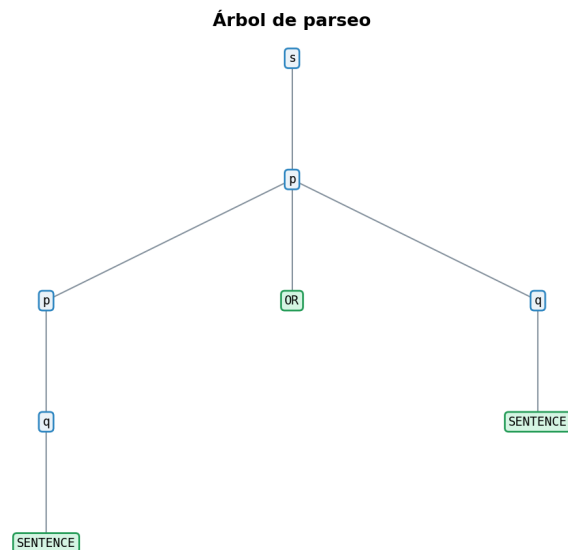


Figura 2. Árbol de parseo de sentence $\vee$ sentence.

10. Construcción LALR(1) por fusión de núcleos

LALR(1) = LR(1) canónico + fusión

1. construir la colección canónica LR(1) (CLOSURE1/GOTO1; cada ítem lleva un lookahead terminal: $[A \rightarrow a \cdot b, t]$).
 2. agrupar estados con el mismo NÚCLEO LR(0) (ítems sin el lookahead).
 3. fusionar cada grupo en un estado: unión de los lookaheads.
- > mismo número de estados que LR(0), con lookaheads precisos.
Reducir $[A \rightarrow a \cdot, t]$ ocurre SÓLO ante t (no ante todo FOLLOW(A)).

Ejemplo de fusión. En el autómata LR(1) de la gramática de ejemplo, dos estados distintos comparten el mismo núcleo LR(0) (difieren solo en el lookahead). Al fusionarlos se unen sus lookaheads en un único estado LALR:

```
LR(1) estado A (#2)
p -> p . OR q , $/AND/OR
s -> p . , $/AND
```

```
LR(1) estado B (#9)
p -> p . OR q , AND/OR/RBRACKET
s -> p . , AND/RBRACKET
```

→ estado LALR fusionado:

```
LALR (A fusionado con B)
p -> p . OR q , $/AND/OR/RBRACKET
s -> p . , $/AND/RBRACKET
```

Para la gramática de ejemplo, LALR produce **12 estados** (igual que LR(0)) y, por ser ya SLR, una tabla idéntica a la SLR. Cada estado muestra los lookaheads fusionados por núcleo:

Autómata LALR(1) — gramática SLR

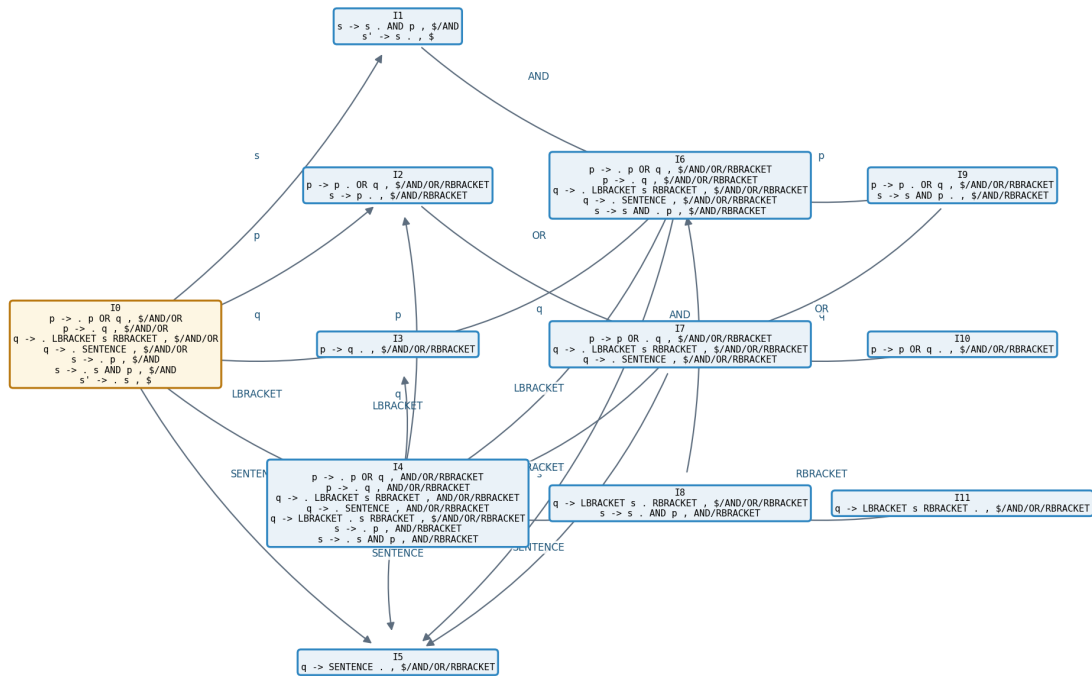


Figura 3. Autómata LALR(1) de la gramática SLR.

11. SLR(1) vs LALR(1): la gramática de punteros

Gramática clásica $S \rightarrow L=R \mid R, L \rightarrow *R \mid id, R \rightarrow L$: es LALR(1) pero **no** SLR(1). SLR genera un conflicto shift/reduce sobre $=$ porque $= \in FOLLOW(R)$; LALR lo evita con lookaheads precisos.

	SLR(1)	LALR(1)
Estados	10	10
Conflictos	1	0

Conflicto detectado por SLR(1):

[shift/reduce] estado I2, símbolo 'ASSIGN': shift -> I6 vs reduce r5 ($r \rightarrow l$)

LALR(1): 0 conflictos. Acepta cadenas como $* id = id$.

Autómata LR(0) — gramática de punteros

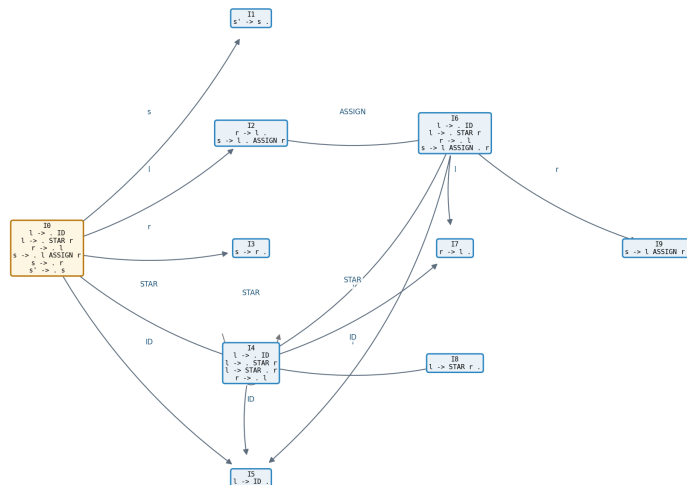


Figura 4. Autómata LR(0) de la gramática de punteros (10 estados).

12. El límite de LALR: una gramática LR(1) que no es LALR(1)

Gramática de Aho-Sethi-Ullman (ej. 4.49): $S \rightarrow aAd \mid bBd \mid aBe \mid bAe$, $A \rightarrow c$, $B \rightarrow c$. Es LR(1), pero al fusionar núcleos los estados $\{A \rightarrow c \cdot\}$ y $\{B \rightarrow c \cdot\}$ (alcanzados por a y por b) se combinan y sus lookaheads $\{d, e\}$ se unen, creando un conflicto **reduce/reduce** que LR(1) canónico no tiene. Es el límite conocido de LALR frente a LR(1).

	SLR(1)	LALR(1)
Estados	13	13
Conflictos	2	2

Conflicto reduce/reduce detectado por LALR(1):

[reduce/reduce] estado I6, símbolo 'TD': reduce r5 (m -> TC) vs reduce r6 (n -> TC)
[reduce/reduce] estado I6, símbolo 'TE': reduce r6 (n -> TC) vs reduce r5 (m -> TC)

Conclusión de la jerarquía: la gramática de punteros (§11) es LALR pero no SLR, y esta es LR(1) pero no LALR. Juntas demuestran **SLR** $\subset$ **LALR** $\subset$ **LR(1)**.

13. Producciones epsilon (gramáticas anulables)

Gramática de expresiones con producciones vacías $e \rightarrow t \ e'$; $e' \rightarrow + \ t \ e' \mid \varepsilon$; $t \rightarrow f \ t'$; $t' \rightarrow * \ f \ t' \mid \varepsilon$; $f \rightarrow (\ e \) \mid \text{id}$. Un ítem $e' \rightarrow \cdot$ está completo de inmediato (genera reducción), y ε se propaga en FIRST. Los FIRST/FOLLOW coinciden con los del libro de texto:

FIRST

e : { ID, LPAREN }
 ep : { PLUS, ε }
 t : { ID, LPAREN }
 tp : { TIMES, ε }
 f : { ID, LPAREN }

FOLLOW

e : { \$, RPAREN }
 ep : { \$, RPAREN }
 t : { \$, PLUS, RPAREN }
 tp : { \$, PLUS, RPAREN }
 f : { \$, PLUS, RPAREN, TIMES }

La gramática es SLR(1) (0 conflictos) y su autómata tiene 16 estados. Acepta entradas como $(x + y) * z$.

Autómata LR(0) — gramática con epsilon

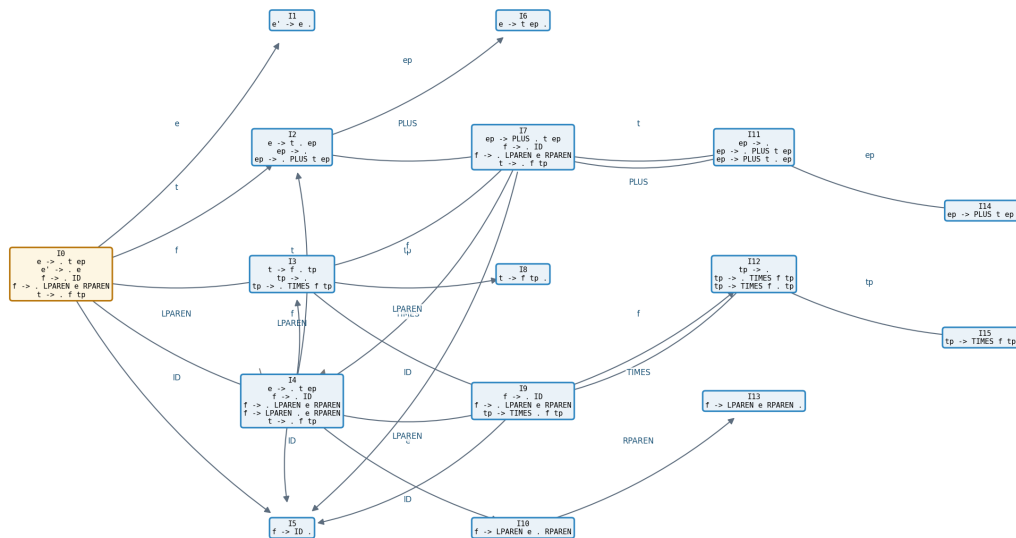


Figura 5. Autómata LR(0) de la gramática con epsilon (16 estados).

14. Manejo de errores

Errores gramaticales (al leer el .yalp): terminal usado sin %token, no-terminal sin definir, nombre de producción en mayúscula, bloque sin :, ausencia de %%. Ejemplos:

- terminal 'Z' used in production 's' is not declared with %token
- non-terminal 'x' used in production 's' is never defined
- missing '%%' separator between token declarations and production rules

Errores sintácticos (al parsear): cuando no hay acción en la celda (estado, token) se reporta el token inesperado, su posición (línea/columna, vía el Token del lexer) y los tokens esperados:

Stack	Input	Acción
0	SENTENCE SENTENCE \$	shift -> I5
0 SENTENCE 5	SENTENCE \$	ERROR (ver mensaje abajo)

CADENA RECHAZADA X
Error sintáctico: no se esperaba el token SENTENCE. Se esperaba uno de: \$, AND, OR, RBRACKET.

15. Uso del programa

```
# Lexer (YALex)
python3 run_generator.py examples/pico/pico.yal -o build/pico_lexer.py

# Parser (YAPar) – flujo completo lexer + gramatica + cadena
python3 run_yapar.py examples/yapar/expr_slr.yalp \
  -l examples/yapar/expr.yal \
  -i examples/yapar/inputs/accept_or.txt --parser both

# Parser autonomo generado
echo 'SENTENCE OR SENTENCE' | python3 build/yapar/expr_slr_slr_parser.py
```

Opciones de run_yapar.py: -l (lexer), -i (entrada), -o (salida), --parser slr|lalr|both, --no-graph. Código de salida: 0 = OK/aceptada, 1 = error/rechazada, 2 = la tabla tiene conflictos.

16. Pruebas y validación

- 12 pruebas de YALex (PICO y ArnoldC; métodos directo y Thompson).
- 39 pruebas de YAPar: gramática, FIRST/FOLLOW, autómatas, tablas, parseo, LALR, epsilon, conflictos (shift/reduce, reduce/reduce, LR(1)-no-LALR) y CLI.
- Smoke test end-to-end (run_smoke_tests.py) de ambos generadores.
- Validación contra el oráculo manual: autómatas de 12 estados, tabla SLR celda por celda y la traza de *sentence* ∨ *sentence*, idénticos.

17. Decisiones técnicas relevantes

- **Numeración estable de estados:** el BFS recorre los símbolos en orden determinista, reproduciendo I0–I11 del documento manual.
- **LALR por fusión de núcleos:** método transparente y fácil de explicar; se construye LR(1) y se fusionan estados con el mismo núcleo LR(0).
- **Resolución de conflictos estilo yacc:** shift sobre reduce; en reduce/reduce, la producción de menor número. Los conflictos se reportan.

- **Lexer pure-ASCII:** las tablas embebidas se emiten con `ascii()` para evitar bytes 0x80–0xFF crudos que rompían el tokenizador de Python.
- **Parser autónomo:** codegen produce un `.py` sin dependencias de yapargen.

Apéndice A. Puntos clave para la presentación

Ideas para defender:

- SLR y LALR tienen los mismos estados (LR(0)); difieren en el lookahead de reduce: FOLLOW(A) global (SLR) vs lookahead del ítem (LALR).
- La jerarquía $SLR \subset LALR \subset LR(1)$ se demuestra con dos gramáticas concretas (punteros y Dragon 4.49).
- Las producciones epsilon se manejan como ítems completos ($A \rightarrow \cdot$) y propagación de ϵ en FIRST/FOLLOW.
- Los conflictos se detectan y reportan; se resuelven estilo yacc para poder continuar.
- La salida (autómata, tabla, traza) coincide con el trabajo hecho a mano, lo que da confianza en la correctitud.

Preguntas probables y respuestas breves:

- ¿Por qué mismos estados? Porque LALR fusiona el LR(1) de vuelta al núcleo LR(0).
- ¿Cómo manejan ϵ ? Ítem $A \rightarrow \cdot$ es reduce inmediato; ϵ se propaga en FIRST.
- ¿Cómo detectan conflictos? Si una celda ya tiene otra acción, se registra (shift/reduce o reduce/reduce) con estado y símbolo.
- ¿Y si la gramática es ambigua? Aparecen conflictos (ej. dangling else); se reportan y se resuelve estilo yacc.
- ¿Cómo se conecta con YALex? Importa el lexer generado, usa Token.type como terminal; IGNORE descarta tokens.

Apéndice B. Referencias y enlaces

- Aho, Lam, Sethi, Ullman. *Compilers: Principles, Techniques, and Tools* (2.^a ed.), cap. 4 (análisis LR, SLR, LALR, FIRST/FOLLOW; ejemplos 4.48 y 4.49).
- Documentación de `ocaml yacc` / `yacc` (formato y resolución de conflictos).
- Documento de la actividad «Construcción del Autómata LR(0)» (oráculo de validación de este informe).
- Repositorio del proyecto: <https://github.com/jlope384/Generador-lex>
- Video de arquitectura (YALex): <https://drive.google.com/drive/folders/16mCpZSuuPGhA4c89ACrT-Dv8gn20exDx>